



The Complete Presentation Guide

From Kernel to Command Line — Everything You Need to Know

■ 30-Minute Deep Dive Presentation

Audience: Beginners with basic awareness of Linux

10 Major Topics

50+ Commands

Real Examples

30 Minutes

■ TABLE OF CONTENTS

1	What is Linux? — History & Philosophy
2	Linux Architecture — Kernel, Shell, Filesystem
3	Linux Distributions — Choosing the Right Distro
4	File System Hierarchy — Where Everything Lives
5	Essential Commands — Navigation & File Operations
6	Users, Groups & Permissions — Security Basics
7	Process Management — Running & Controlling Programs
8	Networking — IP, SSH, Firewall & Tools
9	Package Management — apt, yum, dnf, pacman
10	Shell Scripting — Automate Everything
★	Quick Reference Cheat Sheet

■ 1. WHAT IS LINUX?

History, Philosophy & Why It Matters

What Is Linux?

Linux is a free, open-source operating system kernel first released by Linus Torvalds in 1991. Combined with GNU tools (created by Richard Stallman), it forms a complete OS. Today Linux powers over 96% of the world's top web servers, Android phones, supercomputers, cloud infrastructure, and IoT devices.

Key Milestones

- 1969 — Unix created at Bell Labs (inspiration for Linux)
- 1983 — GNU Project started by Richard Stallman (free software movement)
- 1991 — Linus Torvalds releases Linux kernel v0.01 (wrote it on his PC as a hobby)
- 1994 — Linux kernel 1.0 released publicly
- 2003 — Linux enters enterprise with Red Hat, SUSE
- 2008 — Android (Linux kernel) ships on phones
- 2020 — Microsoft ships Linux kernel inside Windows (WSL2)

Linux Philosophy (The Unix Way)

- Everything is a file (even hardware devices like `/dev/sda`)
- Do one thing and do it well — small focused tools
- Programs work together through pipes (`|`)
- Prefer text streams for data interchange
- Open source — anyone can read, modify, distribute the code

■ Example: Linux is Everywhere!

Your Android phone → Linux kernel

Google, Facebook, Amazon servers → Linux

NASA Mars Rover → Linux (Ingenuity helicopter)

Stock exchange trading systems → Linux

■ ■ 2. LINUX ARCHITECTURE

Layers of the OS Stack

Linux has a layered architecture. Understanding each layer helps you know where to look when something goes wrong.

Layer	Description	Example
Hardware	Physical CPU, RAM, Disk, NIC	Intel Core i7, SSD
Kernel	Core OS — manages hardware resources	linux-6.x
System Libraries	Standard C library, APIs	glibc, libpthread
System Daemons	Background services	systemd, cron, sshd
Shell	Command interpreter	bash, zsh, fish
Applications	User programs	vim, Firefox, gcc

The Kernel — Heart of Linux

- Process Management — schedules which program runs on CPU
- Memory Management — allocates RAM, manages virtual memory & swap
- Device Drivers — lets hardware talk to software
- File System Management — reads/writes files on disk
- Network Stack — handles TCP/IP, sockets, routing
- Security — enforces permissions, SELinux/AppArmor policies

The Shell

The shell is your command-line interface to the kernel. bash (Bourne Again Shell) is the most common default shell.

```
$ echo $SHELL # which shell am I using?
$ cat /etc/shells # list all installed shells
$ chsh -s /bin/zsh # change default shell to zsh
```

■ ■ 3. LINUX DISTRIBUTIONS

Choosing the Right Flavour

A Linux distribution (distro) = Linux kernel + package manager + desktop environment + pre-installed software. There are 300+ distros, but a few families dominate.

Family	Distros	Package Mgr	Best For
Debian	Ubuntu, Mint, Kali, Raspberry Pi OS	apt / dpkg	Beginners, servers, security
Red Hat	RHEL, CentOS, Fedora, Rocky	dnf / rpm	Enterprise, corporate
Arch	Arch Linux, Manjaro, EndeavourOS	pacman	Power users, bleeding-edge
SUSE	openSUSE, SLES	zypper	Enterprise, SAP
Gentoo	Gentoo, Calculate	portage	Extreme customisation
Android	Android (AOSP)	ADB/APK	Mobile devices

My Recommendation for Beginners: Ubuntu

- Huge community — almost every problem has a Stack Overflow answer
- Ubuntu 22.04 LTS — supported until 2027 (LTS = Long Term Support)
- Works on physical PC, VM (VirtualBox/VMware), or WSL2 on Windows
- apt package manager — 60,000+ packages available

■ Example: Check your distro info

```
cat /etc/os-release

uname -a # kernel version + architecture

lsb_release -a # distro name & version
```

■ 4. FILE SYSTEM HIERARCHY

The Linux Directory Tree

Linux uses a single unified tree starting at / (root). There are no C: or D: drives — everything is under /. This is defined by the Filesystem Hierarchy Standard (FHS).

Directory	Purpose	Example Files
/	Root of the entire filesystem	Top of the tree
/bin	Essential user binaries	ls, cp, mv, bash
/sbin	System admin binaries	fdisk, ifconfig, reboot
/etc	System config files	passwd, hosts, fstab, ssh/
/home	User home directories	/home/alice, /home/bob
/root	Root user's home	root's files
/var	Variable data (logs, mail, spool)	/var/log/syslog
/tmp	Temporary files (cleared on reboot)	temp session data
/usr	User programs & libraries	/usr/bin/python3
/opt	Optional/third-party software	Google Chrome install
/proc	Virtual FS — kernel/process info	/proc/cpuinfo
/sys	Virtual FS — hardware/driver info	/sys/class/net/
/dev	Device files	/dev/sda, /dev/null
/boot	Bootloader + kernel images	vmlinuz, grub/
/lib	Shared libraries (.so files)	libc.so.6
/mnt	Temporary mount points	USB drives, NFS
/media	Auto-mounted removable media	CD-ROM, USB

■ Example: Explore the hierarchy

```
ls / # list root directory

ls /etc | head # first 10 config files

df -h # disk usage of each mounted FS

tree /etc -L 1 # tree view (install: apt install tree)
```

■ 5. ESSENTIAL COMMANDS

Navigate, Create, Copy, Move, Search

Navigation

```
pwd # Print Working Directory
ls # list files
ls -la # long listing + hidden files
cd /var/log # change directory to /var/log
cd ~ # go to home directory
cd .. # go up one level
cd - # go back to previous directory
```

File Operations

```
touch report.txt # create empty file
mkdir -p projects/web # create nested directories
cp file.txt backup/ # copy file to backup/
cp -r dir1 dir2 # copy entire directory recursively
mv old.txt new.txt # rename (or move) file
rm file.txt # delete file
rm -rf dirname/ # delete directory and all contents (careful!)
ln -s /etc/hosts hosts_link # create symbolic link
```

Viewing & Editing Files

```
cat /etc/passwd # print file to screen
less /var/log/syslog # scrollable view (q to quit)
head -20 file.txt # first 20 lines
tail -f /var/log/syslog # follow log in real time (Ctrl+C to stop)
wc -l file.txt # count lines in file
nano file.txt # beginner-friendly text editor
vim file.txt # powerful editor (i=insert, :wq=save&quit;)
```

Searching & Finding

```
find / -name 'passwd' # find file named passwd anywhere
find /home -type f -mtime -7 # files modified in last 7 days
grep 'error' /var/log/syslog # search text inside file
grep -r 'TODO' ~/projects/ # recursive search in directory
grep -i 'root' /etc/passwd # case-insensitive search
locate nginx.conf # fast file search (uses index)
which python3 # find where command is installed
```

■ Example: Pipeline Magic

Count how many running processes belong to root:

```
ps aux | grep root | wc -l
```

Find the 5 largest files in current directory:

```
du -sh * | sort -rh | head -5
```


■ 6. USERS, GROUPS & PERMISSIONS

Linux Security Model

Every file in Linux has an owner (user), a group, and a set of permissions. This is the foundation of Linux's multi-user security model.

Understanding Permission Strings

```
-rwxr-xr-- 1 alice developers 4096 May 20 script.sh
| | | | | | | | | | |
| | | | | | | | | | |++-- Others permissions (r-- = read only)
| | | | | | | | | | |++----- Group permissions (r-x = read + execute)
| | | | | | | | | | |++----- Owner permissions (rwx = read+write+execute)
| | | | | | | | | | |++----- File type (- = file, d = directory, l = symlink)
```

Permission Values (Octal)

Symbol	Meaning	Octal
r	Read	4
w	Write	2
x	Execute	1
-	No perm	0

chmod chown chgrp

```
chmod 755 script.sh # rwxr-xr-x (owner=7, group=5, others=5)
chmod u+x script.sh # add execute permission for owner
chmod o-r secret.txt # remove read permission for others
chown alice file.txt # change owner to alice
chown alice:dev file.txt # change owner to alice, group to dev
chgrp developers project/ # change group ownership
```

User Management

```
whoami # current user
id # UID, GID, and group memberships
sudo command # run command as root (superuser)
su - username # switch to another user
useradd -m alice # create new user with home dir
passwd alice # set/change password
usermod -aG sudo alice # add alice to sudo group
userdel -r alice # delete user + home directory
groups alice # show groups alice belongs to
```

■ Example: Secure a sensitive script

```
chmod 700 secret_script.sh # only owner can read/write/execute

chown root secret_script.sh # owned by root

ls -l secret_script.sh # verify permissions
```


■ 7. PROCESS MANAGEMENT

Running & Controlling Programs

A process is a running instance of a program. Each process gets a unique PID (Process ID). Linux is multi-tasking — hundreds of processes run simultaneously.

Viewing Processes

```
ps aux # all processes (a=all, u=user, x=no-tty)
ps aux | grep nginx # find nginx processes
top # interactive real-time process viewer
htop # enhanced top (apt install htop)
pstree # process tree (parent → child)
pidof nginx # get PID of nginx process
pgrep -l ssh # list SSH processes with PIDs
```

Controlling Processes

```
kill 1234 # send SIGTERM (graceful stop) to PID 1234
kill -9 1234 # send SIGKILL (force kill) – last resort
killall nginx # kill all processes named nginx
pkill -f 'python app' # kill by matching command name
nice -n 10 ./script.sh # run with lower priority (10 = less priority)
renice +5 1234 # change priority of running process
```

Background & Foreground Jobs

```
sleep 100 & # run sleep in background (&)
jobs # list background jobs
fg %1 # bring job 1 to foreground
bg %1 # resume stopped job in background
Ctrl+Z # suspend current foreground process
Ctrl+C # terminate current foreground process
nohup ./script.sh & # run even after logout (immune to SIGHUP)
```

systemd — Service Manager

systemd is PID 1 on modern Linux. It manages system services (daemons).

```
systemctl status nginx # check nginx service status
systemctl start nginx # start service
systemctl stop nginx # stop service
systemctl restart nginx # restart service
systemctl enable nginx # auto-start at boot
systemctl disable nginx # remove from boot startup
journalctl -u nginx -f # follow nginx service logs
systemctl list-units --type=service # list all running services
```

■ Example: Monitor system load

```
uptime # load averages (1, 5, 15 minutes)

free -h # RAM usage (h = human readable)
```


■ 8. NETWORKING

IP, SSH, DNS, Firewall & Diagnostics

Network Configuration

```
ip addr show # show all network interfaces and IPs
ip addr show eth0 # show specific interface
ip route show # routing table
ip link set eth0 up/down # bring interface up/down
hostname # show machine hostname
hostname -I # show all IP addresses
cat /etc/hosts # local DNS mappings
cat /etc/resolv.conf # DNS server config
```

Connectivity Diagnostics

```
ping google.com # test connectivity (Ctrl+C to stop)
ping -c 4 8.8.8.8 # ping 4 times then stop
traceroute google.com # trace route to destination
nslookup google.com # DNS lookup
dig google.com # detailed DNS query
curl -I https://google.com # HTTP headers (test web service)
wget https://example.com/file.zip # download file
netstat -tulpn # list listening ports (old tool)
ss -tulpn # list listening ports (modern)
```

SSH — Secure Remote Access

```
ssh user@192.168.1.10 # connect to remote server
ssh -p 2222 user@server # connect on custom port
ssh-keygen -t ed25519 # generate SSH key pair
ssh-copy-id user@server # copy public key to server (no password!)
scp file.txt user@server:/tmp/ # copy file to remote
scp -r dir/ user@server:~/ # copy directory to remote
rsync -avz dir/ user@server:~/dir/ # sync directory efficiently
```

Firewall with UFW (Ubuntu)

```
ufw status # check firewall status
ufw enable # enable firewall
ufw allow 22 # allow SSH
ufw allow 80/tcp # allow HTTP
ufw allow 443/tcp # allow HTTPS
ufw deny 3306 # block MySQL from outside
ufw delete allow 80 # remove a rule
```

■ Example: Quick network troubleshooting

Step 1: Can I reach the internet?

```
ping -c 2 8.8.8.8
```


■ 9. PACKAGE MANAGEMENT

Install, Update & Remove Software

Package managers automate software installation, dependency resolution, and updates. Never manually download .exe files — use your distro's package manager!

apt — Debian / Ubuntu

```
apt update # refresh package list from repos
apt upgrade # upgrade all installed packages
apt install nginx # install nginx
apt install -y git curl wget # install multiple packages, auto yes
apt remove nginx # remove package (keep config)
apt purge nginx # remove package + config files
apt autoremove # remove unused dependencies
apt search 'web server' # search for packages
apt show nginx # show package info
dpkg -l | grep nginx # check if installed
dpkg -i package.deb # install local .deb file
```

dnf / yum — Red Hat / Fedora / CentOS

```
dnf update # update all packages
dnf install httpd # install Apache
dnf remove httpd # remove package
dnf search mysql # search packages
dnf info nginx # package information
rpm -qa | grep python # list installed RPM packages
```

pacman — Arch Linux / Manjaro

```
pacman -Syu # update system
pacman -S vim # install vim
pacman -R vim # remove vim
pacman -Ss 'text editor' # search
pacman -Q # list installed packages
```

Snap & Flatpak — Universal Packages

```
snap install code --classic # install VS Code via snap
snap list # list snap packages
snap remove code # uninstall snap package
flatpak install flathub org.gimp.GIMP # install GIMP
flatpak run org.gimp.GIMP # run flatpak app
```

■ Example: Set up a full LAMP stack in 2 commands

```
sudo apt update && sudo apt install -y apache2 mysql-server php

systemctl start apache2 mysql

# Apache + MySQL + PHP installed and running!
```

■ 10. SHELL SCRIPTING

Automate Everything with Bash

Shell scripting lets you automate repetitive tasks, create system tools, schedule jobs, and build DevOps pipelines. Bash is the most common scripting shell.

Script Basics

```
#!/bin/bash # shebang – tells OS to use bash
# This is a comment

NAME='Alice' # variable (no spaces around =)
echo "Hello, $NAME" # use variable with $
AGE=$(date +%Y) # command substitution
echo "Year: $AGE"
```

Conditionals (if/else)

```
if [ -f /etc/passwd ]; then
echo 'File exists'
elif [ -d /etc ]; then
echo 'Directory exists'
else
echo 'Neither exists'
fi

# Number comparison
NUM=10
if [ $NUM -gt 5 ]; then echo 'Greater than 5'; fi
```

Loops

```
# For loop over list
for fruit in apple banana cherry; do
echo "I like $fruit"
done

# For loop over files
for file in /var/log/*.log; do
echo "Processing: $file"
done

# While loop
count=1
while [ $count -le 5 ]; do
echo "Count: $count"
((count++))
done
```

Functions & Arguments

```
greet() {
echo "Hello, $1! You are $2 years old."
```



```
}  
greet Alice 30 # $1=Alice, $2=30  
  
# Script arguments  
echo "Script: $0"  
echo "First arg: $1"  
echo "All args: $@"  
echo "Arg count: $#"
```

■ Example: Automated backup script

```
#!/bin/bash  
  
BACKUP_DIR='/backup'  
  
DATE=$(date +%Y%m%d_%H%M%S)  
  
SOURCE='/home/alice'  
  
mkdir -p $BACKUP_DIR  
  
tar -czf $BACKUP_DIR/backup_$DATE.tar.gz $SOURCE  
  
echo 'Backup completed: backup_'$DATE'.tar.gz'  
  
# Schedule with cron: crontab -e then add:  
  
# 0 2 * * * /home/alice/backup.sh (runs daily at 2 AM)
```

■ QUICK REFERENCE CHEAT SHEET

Most Used Commands at a Glance

Navigation

```
pwd / ls / cd / tree  
ls -la (hidden + details)  
cd ~ (home) cd - (back)
```

Permissions

```
chmod 755 / chmod u+x  
chown user:group file  
sudo / su / whoami / id
```

Networking

```
ip addr / ping / ss -tulpn  
ssh user@host / scp / rsync  
curl / wget / nslookup / dig
```

Disk & Storage

```
df -h / du -sh * / lsblk  
mount / umount / fdisk -l  
tar -czf / tar -xzf
```

Scripting

```
#!/bin/bash (shebang)  
if/elif/else/fi for/do/done  
crontab -e (schedule tasks)
```

Files

```
touch / mkdir / cp / mv / rm  
cat / less / head / tail / wc  
find / grep / locate / which
```

Processes

```
ps aux / top / htop / pstree  
kill PID / kill -9 / killall  
systemctl start/stop/status
```

Packages (apt)

```
apt update && apt upgrade  
apt install PKG / apt remove PKG  
apt search / apt show
```

Text Processing

```
grep / sed / awk / cut / sort  
uniq / tr / tee / xargs  
diff file1 file2 / patch
```

System Info

```
uname -a / lscpu / lsmem  
free -h / uptime / vmstat  
dmesg / journalctl / last
```

Thank You! ■ Keep Exploring Linux — man is your best friend.

Useful resources: linux.org | man7.org | tldr.sh | explainshell.com